

**ASIA PACIFIC UNIVERSITY OF TECHNOLOGY & INNOVATION****CT101-3-3-IOT- INDIVIDUAL ASSIGNMENT REPORT**

TITLE	SMART PARKING SYSTEM
NAME & STUDENT ID	NOOR UR RASHID (TP066720)
INTAKE	APD4F2601CE
LECTURER	KAMALANATHAN A/L SHANMUGAM
Date	27/04/2026

Table of Contents

1.0 Introduction.....	4
1.1 Introduction to Arduino.....	4
1.2 Introduction to Project	4
2.0 System Component	5
3.0 System Design	7
3.1 Step 1: Arduino Uno and RFID RC522 Connection.....	7
3.2 Step 2: Arduino Uno and I2C LCD 16x2 Connection	7
3.3 Step 3: Arduino Uno and Ultrasonic Sensor 1 Connection.....	8
3.4 Step 4: Arduino Uno and Ultrasonic Sensor 2 Connection.....	8
3.5 Step 5: Arduino Uno and Servo Motor Connection.....	9
3.6 Step 6: Arduino Uno and LED Indicators Connection	9
3.7 Step 7: Breadboard Power Distribution	10
3.8 Step 8: Overall System Operation.....	10
4.0 Prototype	11
5.0 Source Code	16
5.1 Program Initiation	16
5.2 Program Setup.....	17
5.3 Loop Operation	18
5.4 Functions.....	20
6.0 Limitation.....	21
7.0 Future Enhancement	21
8.0 Conclusion	22
9.0 Prototype Circuit Link	22
10.0 References.....	23

List of Figures

Figure 1 Smart Parking System Simulation.....	10
Figure 2 Smart Parking System Prototype.....	12
Figure 3 Smart Parking System Parking Slot Availability	12
Figure 4 Smart Parking System Gate Opening using RDIF	13
Figure 5 Smart Parking System Slot 1 Full	13
Figure 6 Smart Parking System Slot 2 Full	14
Figure 7 Smart Parking System Both Slot are Full.....	14
Figure 8 Smart Parking System RFID Gate Access Denied	15
Figure 9 Program Initiation.....	16
Figure 10 Program Setup	17
Figure 11 Loop Operation.....	19
Figure 12 Functions	20

1.0 Introduction

1.1 Introduction to Arduino

Arduino is an open-source electronics platform that is widely used for developing embedded and IoT-based systems. It is founded on both hardware and software that is easy to use and is accessible to the novice yet has the flexibility to support advanced uses. There are many Arduino boards that include Arduino Uno, Mega, and Nano, which are designed to meet various project needs. Arduino has a significant role in the current technology, particularly in the field of automation, smart systems, in industry, and in the development of IoT. Using Arduino, it is easy to combine sensors and actuators to form smart systems capable of monitoring and controlling real worlds Arduino (2023).

1.2 Introduction to Project

Parking management is now a big problem in the contemporary urban setting because of the rising number of vehicles. Ineffective parking systems usually lead to traffic jams, loss of time and wastage of space. To overcome such problems, a Smart Parking System with Arduino has been designed. RFID technology is applied in this project to identify the vehicles and ultrasonic sensors are applied to identify the availability of parking slots. A servo motor is introduced to drive the gate mechanism, LEDs and LCD display are introduced to give real time feedback of the system to the users. The system will enhance the efficiency in the parking system and minimize congestion and ease of use of the system Tanti et al. (2020).

2.0 System Component

Component	Description	Role
Arduino Maker Uno	An ATmega328P based microcontroller board with built-in indicators to facilitate easy prototyping and learning.	It is used as the main controller where all sensor data is processed and decisions are executed to control outputs such as the servo, LEDs, and LCD.
RFID RC522 Module	Radio frequency identification module, which reads RFID card identification numbers (UID) on RFID cards.	It is employed to verify users. The scanned UID is compared with the stored authorized data to give access or deny access.
Ultrasonic Sensor (HC-SR04)	Sensors that measure distance by sending ultrasonic waves and measuring distance.	They are applied to establish whether parking spaces are in use or not by measuring the distance between the sensor and the vehicle.
Servo Motor	A positional motor which can rotate to particular angles with respect to control signals.	It is employed to operate the parking gate by opening and closing it when they are allowed to access or after some delay.
LEDs	Components which emit light when a current is passed through them.	They can be utilized as indicators with green indicating an available slot and red indicating an occupied slot.
I2C LED 16x2 Display	A display module that indicates text data via I2C communications with less wiring.	It is utilized in indicating parking status, available slots and system messages like access granted and access denied.
Resistors	Passive electronic devices that are used to restrict current in a circuit.	They are applied to shield LEDs against overcurrent as well as to

		provide safe operation of the circuit.
Breadboard	A circuit board with a reusable prototyping board with no soldering.	It is utilized to join all the components in a well-organized way and enables easy adjustments during the development.
Jumper Wires	Prototyping tools to solder components.	They are employed to construct and structure the circuit connection of all components as well as to the microcontroller.

3.0 System Design

3.1 Step 1: Arduino Uno and RFID RC522 Connection

The RFID RC522 module is connected to the Arduino Uno, in the first step, to be used to perform user authentication. The RFID module will be supplied with 3.3 V pin and ground pin on the Arduino Uno. Communication between the RFID module and the Arduino Uno is done using the SPI communication. The RFID module sda pin is linked to pin 10 of the Arduino Uno. SCK pin is connected to digital pin 13, MOSI pin is to digital pin 11 and MISO pin is to digital pin 12. Also, the RST pin in the RFID module is attached to digital pin 9. Via this interconnection, the microcontroller is able to read the RFID card data and compare it to the authorized UID stored in the program.

RFID RC522 Pin	Connection
SDA	Arduino Uno Maker Digital Pin 10
SCK	Arduino Uno Maker Digital Pin 13
MOSI	Arduino Uno Maker Digital Pin 11
MISO	Arduino Uno Maker Digital Pin 12
RST	Arduino Uno Maker Digital Pin 9
VCC	Arduino Uno Maker 3.3V
GND	Arduino Uno Maker GND

3.2 Step 2: Arduino Uno and I2C LCD 16x2 Connection

The second step involves connecting the I2C LCD 16x2 display to the Arduino Uno in order to give feedback to the user in the form of a display. The Arduino Uno 5V and GND pins are used to power the LCD. The I2C modules are employed, and therefore, only two communication lines can be provided, which assists in saving a number of pins and making the wiring simpler. The SDA pin of the LCD is set to the analog pin A4 of the arduino Uno and the SCL pin is set to analog pin A5. This connection allows displaying information like slot availability, access status and parking messages.

LCD Pin	Connection
GND	Arduino Uno Maker GND
VCC	Arduino Uno Maker 5V
SDA	Arduino Uno Maker A4
SCL	Arduino Uno Maker A5

3.3 Step 3: Arduino Uno and Ultrasonic Sensor 1 Connection

The third step is to have a first ultrasonic sensor attached to the Arduino Uno that will sense the occupancy of the Parking Slot 1. The sensor is connected to the Arduino Uno power supply (5 V, and GND) through the breadboard. The Ultrasonic Sensor 1 Trig pin is joined to the digital pin 2 and Echo pin is joined to digital pin 3. The distance between the sensor and an object is determined by sending and receiving ultrasonic signals. When short distance is detected then the parking slot is regarded to be occupied. When a greater distance is observed, then the slot is vacant.

Ultrasonic Sensor 1 Pin	Connection
VCC	Arduino Uno Maker 5V
GND	Arduino Uno Maker GND
Trig	Arduino Uno Maker Digital Pin 2
Echo	Arduino Uno Maker Digital Pin 3

3.4 Step 4: Arduino Uno and Ultrasonic Sensor 2 Connection

The second ultrasonic sensor is attached to the Arduino Uno in the fourth step so that the occupancy status of Parking Slot 2 can be detected. Like the first ultrasonic sensor, the 5V and GND pins of the Arduino Uno are used to supply power to the breadboard. Ultrasonic Sensor 2 has Trig pin that is connected to digital pin 4, and Echo pin that is connected to digital pin 5. The sensor is utilized to keep track of the second parking slot. The parking space availability is updated according to the distance measured.

Ultrasonic Sensor 2 Pin	Connection
VCC	Arduino Uno Maker 5V

GND	Arduino Uno Maker GND
Trig	Arduino Uno Maker Digital Pin 4
Echo	Arduino Uno Maker Digital Pin 5

3.5 Step 5: Arduino Uno and Servo Motor Connection

The fifth step includes the connection of the servo motor to the Arduino Uno where the motor controls the parking gate opening and closing. The supply of the servo motor is 5V and GND. The signal wire of the servo motor is connected to digital pin 6 of the Arduino Uno. On detecting a valid RFID card and having at least one parking slot free, the servo motor is turned to a particular angle in order to open the gate. The servo motor is rotated once again to its starting point after a brief delay to close the gate. By doing so, automated barrier control is accomplished.

Servo Motor Wire	Connection
VCC	Arduino Uno Maker 5V
GND	Arduino Uno Maker GND
Signal	Arduino Uno Maker Digital Pin 6

3.6 Step 6: Arduino Uno and LED Indicators Connection

In the sixth step, four LEDs are connected to the Arduino Uno to indicate the status of the two parking slots. Each parking slot has a green LED and a red LED. A green LED indicates a slot that is available and a red LED indicates an occupied slot. Parking Slot 1: In this case, the green LED is wired to digital pin 7 and the red LED is wired to digital pin 8. In the case of Parking Slot 2, the green LED is connected to A0 analog pin and the red LED is connected to A1 analog pin. The LEDs are put in series with current-limiting resistors in order to ensure that they are not subjected to too much current.

LED	Connection
Slot 1 Green LED	Arduino Uno Maker Digital Pin 7
Slot 1 Red LED	Arduino Uno Maker Digital Pin 8
Slot 2 Green LED	Arduino Uno Maker Analog Pin A0

Slot 2 Red LED	Arduino Uno Maker Analog Pin A1
----------------	---------------------------------

3.7 Step 7: Breadboard Power Distribution

The seventh step involves giving power and ground connections to all components by distributing them using the breadboard. The power rail of the breadboard is connected to 5V pin of the Arduino Uno, and the ground rail is connected to the GND pin of the Arduino Uno. The power is fed to the ultrasonic sensors, LCD module, servo motor, and LEDs through the breadboard. This is a more organized circuit and enables the circuit connections to be made with greater ease without the need to solder.

3.8 Step 8: Overall System Operation

Once all the components are interconnected, the system will work as an integrated Smart Parking System. The RFID reader authenticates the user and only then is the user given access to the gate. Simultaneously, the ultrasonic sensors constantly watch the parking slots in order to know whether they are filled or not. The LEDs indicate the parking slot status and also on the LCD display. In case of a valid RFID card, when scanned, and there is at least one parking slot available, the servo motor unlocks the gate. When parking spaces are full, they will not allow access even with a valid RFID card. By so doing, security as well as parking management capabilities is attained under one system.

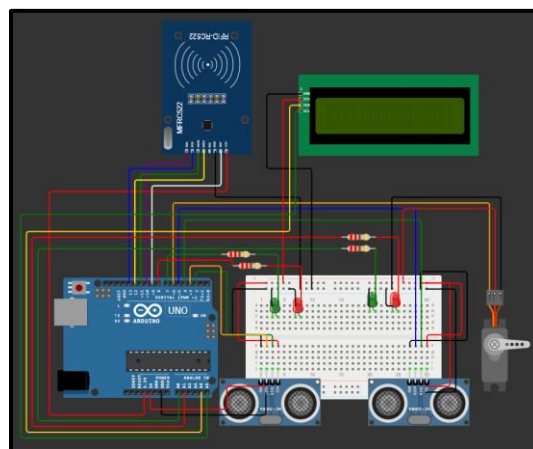
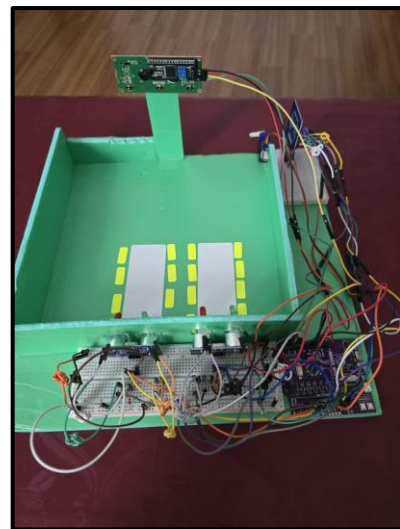
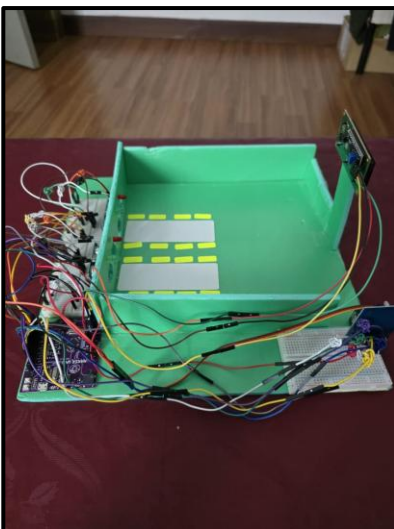
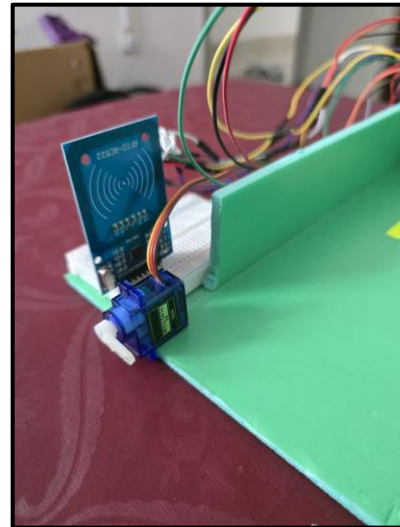
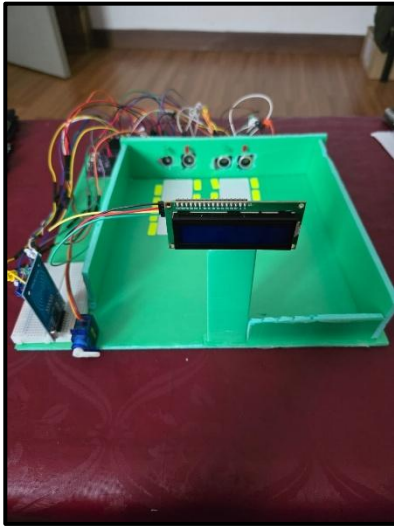


Figure 1 Smart Parking System Simulation

4.0 Prototype



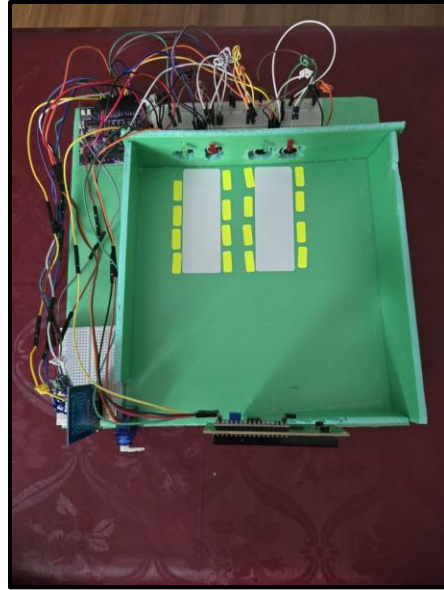


Figure 2 Smart Parking System Prototype

The overall prototype of the Smart Parking System is presented.

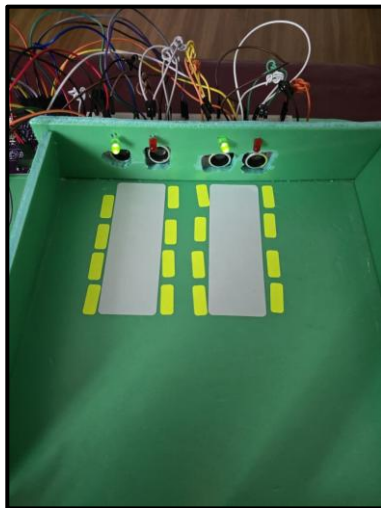


Figure 3 Smart Parking System Parking Slot Availability

The LCD display shows the availability of parking slots and the ultrasonic sensors do not detect a vehicle.

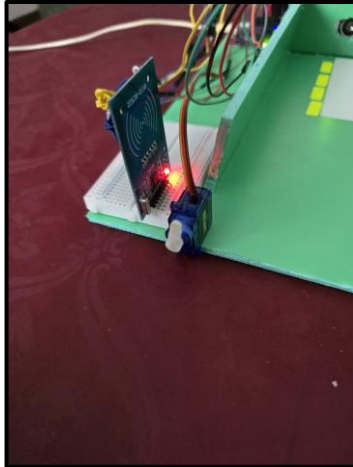


Figure 4 Smart Parking System Gate Opening using RDIF

The RFID card will be used to control the access of the gate and once access is given, a message is shown on the LCD screen. Only when there are parking slots, the gate is opened.

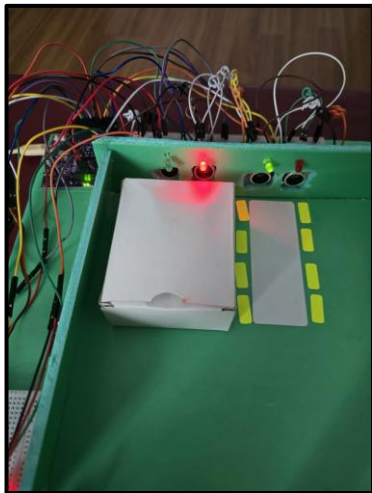


Figure 5 Smart Parking System Slot 1 Full

The ultrasonic sensor detects that Parking Slot 1 is occupied and the new number of available slots is displayed on the LCD.

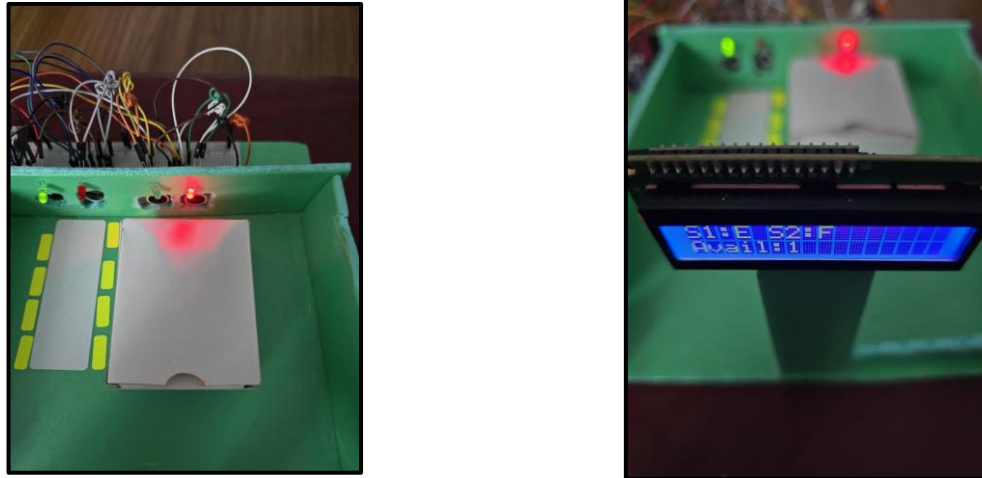


Figure 6 Smart Parking System Slot 2 Full

When the ultrasonic sensor detects parking Slot 2 occupied, the new parking availability is displayed on the LCD device.



Figure 7 Smart Parking System Both Slot are Full

The ultrasonic sensors detect this condition when both the parking slots are occupied and the LCD shows that there are no parking slots available.

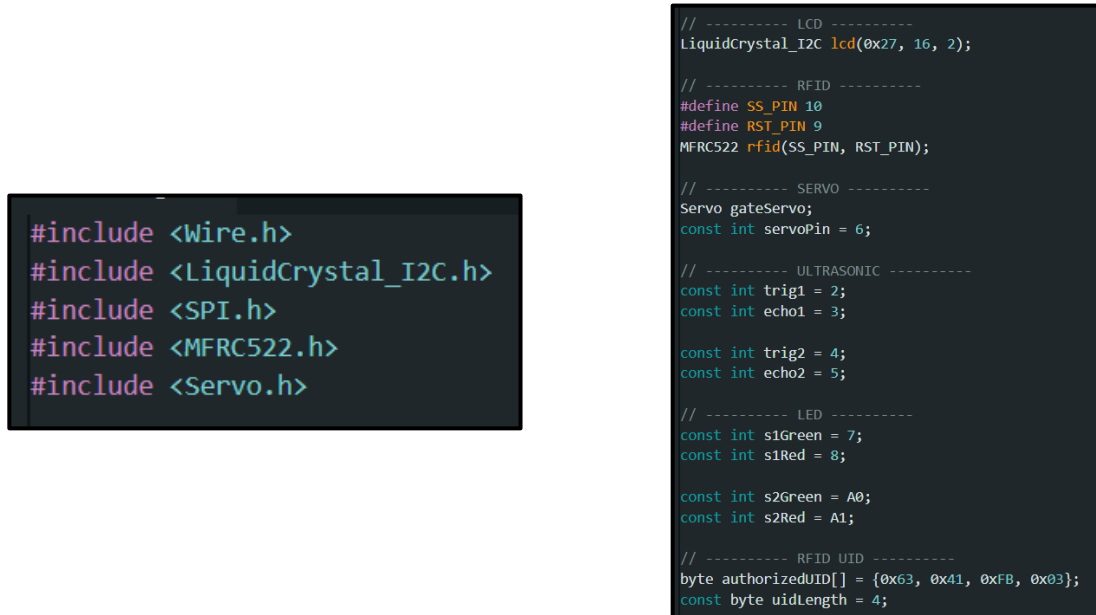


Figure 8 Smart Parking System RFID Gate Access Denied

Any effort to gain access with an RFID card when all the parking slots are occupied will lead to a message being displayed on the LCD that states that the parking is full and it denies access.

5.0 Source Code

5.1 Program Initiation



```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <SPI.h>
#include <MFRC522.h>
#include <Servo.h>
```

```
// ----- LCD -----
LiquidCrystal_I2C lcd(0x27, 16, 2);

// ----- RFID -----
#define SS_PIN 10
#define RST_PIN 9
MFRC522 rfid(SS_PIN, RST_PIN);

// ----- SERVO -----
Servo gateServo;
const int servoPin = 6;

// ----- ULTRASONIC -----
const int trig1 = 2;
const int echo1 = 3;

const int trig2 = 4;
const int echo2 = 5;

// ----- LED -----
const int s1Green = 7;
const int s1Red = 8;

const int s2Green = A0;
const int s2Red = A1;

// ----- RFID UID -----
byte authorizedUID[] = {0x63, 0x41, 0xFB, 0x03};
const byte uidLength = 4;
```

Figure 9 Program Initiation

The program has a number of libraries at the start of the program to assist the elements of the system. LCD communication is provided by the Wire.h and LiquidCrystal_I2C.h libraries, RFID functionality is provided by SPI.h and MFRC522.h. Servo.h library is provided to control servo motor. Then, LCD, RFID module, and servo motor objects are initialized. Pins to use in ultrasonic sensors, LEDs, and the servo motor are all defined. The authentication is also stored in the authorized RFID UID. Moreover, there are variables that are defined to hold values of distance and parking slot status.

5.2 Program Setup

```
// ----- SETUP -----  
void setup() {  
  Serial.begin(9600);  
  
  SPI.begin();  
  rfid.PCD_Init();  
}
```

```
// LCD  
lcd.init();  
lcd.backlight();  
  
// Servo  
gateServo.attach(servoPin);  
gateServo.write(0);
```

```
// Pins  
pinMode(trig1, OUTPUT);  
pinMode(echo1, INPUT);  
pinMode(trig2, OUTPUT);  
pinMode(echo2, INPUT);  
  
pinMode(s1Green, OUTPUT);  
pinMode(s1Red, OUTPUT);  
pinMode(s2Green, OUTPUT);  
pinMode(s2Red, OUTPUT);  
  
lcd.setCursor(0, 0);  
lcd.print("Smart Parking");  
lcd.setCursor(0, 1);  
lcd.print("Initializing...");  
delay(2000);  
lcd.clear();  
}
```

Figure 10 Program Setup

All the components are initialized in the setup() function. A serial communication is initiated to debug. The RFID module and SPI communication are started. This is followed by the initiation of the LCD display and the backlight is activated. Servo motor is fitted and connected to a starting position. PinMode is used to configure all input and output pins. Lastly, the LCD screen displays a startup message, which means that the system is ready to be used.

5.3 Loop Operation

```
// ----- LOOP -----  
void loop() {  
  
    // Read distances  
    dist1 = getDistance(trig1, echo1);  
    dist2 = getDistance(trig2, echo2);  
  
    // Slot logic  
    slot1Free = (dist1 > 10);  
    slot2Free = (dist2 > 10);
```

```
    // LCD display  
    lcd.setCursor(0, 0);  
    lcd.print("S1:");  
    lcd.print(slot1Free ? "E" : "F");  
    lcd.print(" S2:");  
    lcd.print(slot2Free ? "E" : "F");  
    lcd.print(" ");  
  
    lcd.setCursor(0, 1);  
    lcd.print("Avail:");  
    lcd.print(availableSlots);  
    lcd.print(" ");
```

```
    // LED control  
    digitalWrite(s1Green, slot1Free);  
    digitalWrite(s1Red, !slot1Free);  
  
    digitalWrite(s2Green, slot2Free);  
    digitalWrite(s2Red, !slot2Free);  
  
    int availableSlots = slot1Free + slot2Free;
```

```
    // ----- RFID CHECK -----  
    if (!rfid.PICC_IsNewCardPresent()) return;  
    if (!rfid.PICC_ReadCardSerial()) return;
```

```

Serial.println("Card Scanned");
Serial.print("UID: ");
for (byte i = 0; i < rfid.uid.size; i++) {
  if (rfid.uid.uidByte[i] < 0x10) {
    Serial.print("0");
  }
  Serial.print(rfid.uid.uidByte[i], HEX);
  Serial.print(" ");
}
Serial.println();

if (isAuthorized(rfid.uid.uidByte, rfid.uid.size)) {

  if (availableSlots > 0) {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Access Granted");
    lcd.setCursor(0, 1);
    lcd.print("Opening Gate");

    openGate();
  } else {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Parking FULL");
    lcd.setCursor(0, 1);
    lcd.print("Access Denied");
    delay(2000);
  }

} else {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Invalid Card");
  lcd.setCursor(0, 1);
  lcd.print("Access Denied");
  delay(2000);
}

lcd.clear();

rfid.PICC_HaltA();
rfid.PCD_StopCrypto1();

```

Figure 11 Loop Operation

The system is continuously run in the looping function of the system. The ranges to both ultrasonic sensors are calculated to know the position of each parking slot. When the distance exceeds a specified limit the slot is said to be available, otherwise it is occupied. The LEDs are updated depending on the slot status and the amount of slots available is shown on the LCD. The RFID card is subsequently verified. In case of no card, the system will keep on monitoring. On scanning a card, UID is read and checked. Should the card be validated and there is a parking lot, the gate is opened and access is allowed. When the parking is occupied or the card is invalid the access is denied and message is shown on the LCD.

5.4 Functions

```
// ----- FUNCTIONS -----  
  
float getDistance(int trig, int echo) {  
    long duration;  
    float distance;  
  
    digitalWrite(trig, LOW);  
    delayMicroseconds(2);  
  
    digitalWrite(trig, HIGH);  
    delayMicroseconds(10);  
    digitalWrite(trig, LOW);  
  
    duration = pulseIn(echo, HIGH, 30000);  
  
    if (duration == 0) return 999;  
  
    distance = duration * 0.034 / 2;  
    return distance;  
}
```

```
bool isAuthorized(byte *uid, byte size) {  
    if (size != uidLength) return false;  
  
    for (byte i = 0; i < uidLength; i++) {  
        if (uid[i] != authorizedUID[i]) return false;  
    }  
    return true;  
}
```

```
void openGate() {  
    gateServo.write(90);  
    delay(4000);  
    gateServo.write(0);  
}
```

Figure 12 Functions

Three main functions are used in the program. The ultrasonic sensors are applied in the `getDistance()` function to measure the distance. The `isAuthorized()` method is a method that is utilized to check whether the scanned RFID UID is the same as the stored authorized UID. The servo motor is controlled by use of the `openGate()` function. The rotation of the servo to a certain angle opens the gate and the closing is performed after some delay.

6.0 Limitation

In the Smart Parking System developed, several limitations are found. To begin with, there is only one gate at the entrance and exit, and this could create a bottleneck when more than two vehicles are trying to reach the parking area at the same time. This drawback makes it less efficient when it comes to real-life applications that have increased traffic.

Second, the RFID authentication system works on a predetermined UID, which is entrenched in the program. Consequently, there is a lack of flexibility and even new users cannot be dynamically added without an amend to the code. This reduces the scalability of the system to larger implementations.

Moreover, the ultrasonic sensors can only sense the existence of objects depending on distance. They cannot differentiate between various kinds of objects and this could result in misdetection in case of presence of unwanted objects in the sensing range.

In addition, the system lacks data storage and communication facility. Thus, parking information like history of usage and user information cannot be remotely documented or tracked.

Finally, the prototype only has two parking slots, making it only applicable to small-scale environments. In practice, more sensors and more sophisticated integration of the system would be necessary.

7.0 Future Enhancement

The Smart Parking System can be optimized by several enhancements to improve its functionality and performance. To begin with, the system may be advanced with a more developed microcontroller like ESP32 that will enable it to connect to IoT platforms. With this upgrade, it is possible to perform remote monitoring and control with mobile applications or web interfaces.

Secondly, a database system may be implemented to take care of the RFID users at dynamic levels. This would enable a variety of users to be added, deleted or changed without changing the program code and hence enhance the flexibility and scalability of the system.

Moreover, further sophisticated sensors like camera-based systems or object detection using AI can also be introduced to enhance the accuracy of parking slot detection. This would assist in minimizing mistakes brought about by undesired objects.

Moreover, entrance and exit gates may be divided to enhance the movement of traffic and eliminate congestion. The gate can also have additional sensors that will detect the movement of vehicles automatically without necessarily using RFID.

Lastly, more sensors can be added to the system to support more parking slots and optimize the control system. The ability to reserve parking in real-time, mobile notifications, payment integration may also be added to provide additional convenience to users and turn the system into a full-fledged smart parking system.

8.0 Conclusion

Finally, a Smart Parking System using the Maker UNO (Arduino-compatible board) has been developed and proven. The system combines RFID authentication, ultrasonic sensors and LED indicators with LCD display and a servo motor to offer an automated parking solution. This integration provides real-time monitoring of availability of parking slots, and allows efficient control of access to the parking area. The project objectives have been met, since the system can identify the occupancy of parking slots, show the slot availability and regulate the gate upon user authentication and space availabilities. They have successfully used Arduino technology, which underscores its role in the creation of smart systems and automation solutions of the present day. Despite some of its limitations, including the use of a single gate and limited scalability, the prototype forms a solid base on which the improvements can be made in the future. In general, the project illustrates the applicability of simple embedded systems to real-life issues in the urban setting, especially in enhancing parking facilities and minimizing inefficiencies.

9.0 Prototype Circuit Link

The prototype complete circuit is shown as Wokwi simulator link without functionality.

<https://wokwi.com/projects/461009180595909633>

10.0 References

Arduino. (2023, March 9). *What is Arduino? | Arduino Documentation*. Docs.arduino.cc.
<https://docs.arduino.cc/learn/starting-guide/whats-arduino>

Jaafar Ahmed Abdulsahab, Raghad Mohanned Nafea, Ameen, W., & Mohammed Lateef Hayyaw. (2024). IoT Based Smart Parking System. *Journal Port Science Research*, 7(3).
<https://doi.org/10.36371/port.2024.3.1>

Tanti, H., Kasodariya, P., Patel, S., & Rangrej, D. H. (2020). Smart Parking System based on IOT. *International Journal of Engineering Research And*, V9(05).
<https://doi.org/10.17577/ijertv9is050041>

Smart car parking system. (n.d.). Wwww.youtube.com.
<https://www.youtube.com/shorts/wCqskL2kZ-o>